PROG5 (GBIN5U03) - Examen de seconde session
Année 2025-2026

Durée : 2h.
Tout document ou calculatrice interdit. Un dictionnaire bilingue non annoté pour les étudiants étrangers. Pour chaque question, une partie des points peut tenir compte de la présentation.
Le barême est indicatif.

UFR IM²AG

UNIVERSITÉ Grenoble Alpes

1 Echauffement en C (3 points)

Dans cette partie, nous supposons qu'un entier correspond au type `uint32_t`.

1. (1,5 point) Ecrire une fonction qui détermine si un entier qui lui est passé en paramètre aura la même valeur en mémoire en little endian et en big endian.

2. (1,5 point) Ecrire une fonction qui compte le nombre de bits à 1 d'un entier passé en paramètre.

2 Shell (3 points)

Nous supposons que le répertoire courant contient deux sous répertoires nommés `Entrees` et `Sorties`. Ecrire un script qui, pour chaque fichier `x` contenu dans le répertoire `Entrees` :

- exécute le programme dont le nom est fourni en premier argument du script, en redirigeant le contenu de `x` comme entrées de ce programme ;

- si les sorties affichées lors de cette exécution sont identiques au contenu du fichier `x` du repertoire `Sorties`, affiche `SUCCES`, sinon, affiche `ECHEC`.

À la fin de son exécution, le script devra renvoyer 1 si au moins un des tests a échoué et 0 sinon.

3 Makefile (4 points)

Dans cette partie, nous utiliserons les fonctions suivantes, vues en TP, dont nous rappelons ici brièvement le fonctionnement :

- `$(wildcard motif)` : est remplacé par l'ensemble des fichiers du répertoire courant dont le nom correspond au motif (le motif est interprété au sens du shell, des exemples de motifs sont `*` ou `*.c`) ;

- `$(patsubst motif, substitut, liste)` : affiche la liste de mots donnée en remplaçant les occurences du motif par le substitut donné. Dans le motif, si le caractère `%` apparaît, sa première occurrence est traitée comme un joker : elle peut correspondre à toute sous séquence de caractère et cette sous séquence est réutilisée à la place du premier caractère `%` dans le substitut (s'il y a lieu).

Questions :

1. (1 point) donner la règle générique utilisée par `make` pour la fabrication des `.o` à partir des `.c`.

2. (1,5 points) écrire un `Makefile` qui permet d'utiliser la commande `make` avec un nom de programme `p` en argument. Les règles de ce `Makefile` devront faire en sorte que `make` :

   - compile le programme donné a partir du fichier source `p.c` ;
   - teste le programme résultant à l'aide du script de la partie 2 si l'un des fichiers du répertoire `Entrees` a changé.

3. (1,5 points) étendre ce `Makefile` afin que, pour tout fichier `.c` du repertoire courant, le même traitement soit effectué sur le programme associé.

4 Surcouche d'entrées/sorties (10 points)

Dans cette partie, vous devez écrire un ensemble de fonctions permettant d'afficher du texte dans un flux de sortie sous une forme améliorée :

- chaque ligne devra être affichée précédée de son numéro (le numéros débutent à 1 et iront au plus jusqu'à 999) affiché lui-même sur 3 caractères et qui devra être suivi des caractères deux points et espace. Rappel : les directives `%` de printf peuvent être suivies d'un entier indiquant le nombre de colonnes sur lequel afficher ;

- chaque ligne devra être coupée au bout de 80 caractères (numéro compris). Si une ligne est strictement plus longue que 80 caractères, le reste de la ligne devra être affiché sur la ligne suivante sans numéro mais indenté de 5 espaces. Si ce reste de ligne est lui-même plus long que 80 caractères il subira le même traitement, et ainsi de suite ;

- les tabulations devront être remplacées par 4 espaces ;

- le caractère `'$'` et le nom de variable qui suit doivent être remplacés par la valeur de la variable d'environnement correspondante. Nous nous limiterons aux noms de variable composés uniquement de caractères alphabétiques. Pour obtenir la valeur d'une variable d'environnement, vous pourrez utiliser la fonction `getenv`, qui renvoie la valeur de la variable de nom donné et dont le prototype est le suivant :

  ```
  char * getenv(const char *name);
  ```

L'ensemble de fonctions réalisant l'affichage au format demandé aura la spécification donnée dans le fichier `my_print.h` dont le contenu est retranscrit ci-dessous :

```c
#ifndef __MY_PRINT_H__
#define __MY_PRINT_H__
#include "my_print_type.h"

/* my_print_init
 * Description : initialise les structures de données utilisées par l'ensemble
 *               des fonctions my_print_* pour gérer un flux formatté
 * Arguments : un flux dans lequel il est possible d'écrire
 * Valeur de retour : un pointeur vers MY_PRINT_DATA (flux formatté initialisé)
                      ou NULL en cas d'erreur
 * Effets de bord : peut necessiter une allocation mémoire
 */
MY_PRINT_DATA *my_print_init(FILE *);

/* my_print_char
 * Description : affiche un caractère dans un flux formatté donné
 * Arguments : un flux formatté initialisé et un caractère
 * Valeur de retour : aucune
 * Effets de bord : écrit dans le flux associé au flux formatté donné
 */
void my_print_char(MY_PRINT_DATA *, char);

/* my_print_string
 * Description : affiche une chaîne de caractères dans un flux formatté donné
 * Arguments : un flux formatté initialisé et une chaîne de caractères
 * Valeur de retour : aucune
 * Effets de bord : écrit dans le flux associé au flux formatté donné
 */
void my_print_string(MY_PRINT_DATA *, char *);

/* my_print_terminate
 * Description : libère les ressources associées à un flux formatté donné
 * Arguments : un flux formatté initialisé
 * Valeur de retour : aucune
 * Effets de bord : peut libérer de la mémoire, peut écrire dans le flux
 *                  associé au flux formatté donné
 */
void my_print_terminate(MY_PRINT_DATA *);

#endif
```

A titre d'exemple, le programme suivant :

```c
#include "my_print.h"

int main() {
    MY_PRINT_DATA *p;

    p = my_print_init(stdout);
    my_print_string(p, "Voici mon HOME : $HOME\n");
    my_print_string(p, "Voici de nouveau mon HOME sur une ligne vraiment très très "
                       "longue : ");
    my_print_char(p, '$');
    my_print_char(p, 'H');
    my_print_char(p, 'O');
    my_print_char(p, 'M');
    my_print_char(p, 'E');
    my_print_char(p, '\n');
    my_print_string(p, "\tUne tabulation\n");
    my_print_string(p, "\t\tDeux tabulation\n");
    my_print_terminate(p);
    return 0;
}
```

Doit produire l'affichage suivant :

```
1: Voici mon HOME : /home/zebulon
2: Voici de nouveau mon HOME sur une ligne vraiment très très longue : /home
   /zebulon
3:     Une tabulation
4:         Deux tabulation
```

Questions :

1. (1 point) Ecrire le contenu du fichier `my_print_type.h` contenant la déclaration du type abstrait `MY_PRINT_DATA` utilisé dans `my_print.h`. Commentez votre réponse afin d'expliciter l'utilisation qui sera faite de chaque constituant du type `MY_PRINT_DATA`.

2. (6 points) Ecrire le contenu du fichier `my_print.c` contenant l'implémentation des fonctions décrites dans `my_print.h`.

3. (3 points) Ecrire un programme qui affiche à l'aide des fonctions `my_print_*` que vous venez d'écrire le contenu d'un fichier dont le nom est donné en argument de la ligne de commande (ou le contenu de l'entrée standard si aucun nom n'est donnée).

Remarque : la question 3 peut être traitée sans avoir répondu aux questions 1 et 2